

Parallel & Distributed Computing



DISTRIBUTED SHARED MEMORY



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

2

"إِنَّ مَعَ الْعُسْرِ يُسْرًا"

"Indeed, with hardship
there is ease"

(94:6)

Lecture Outline

3

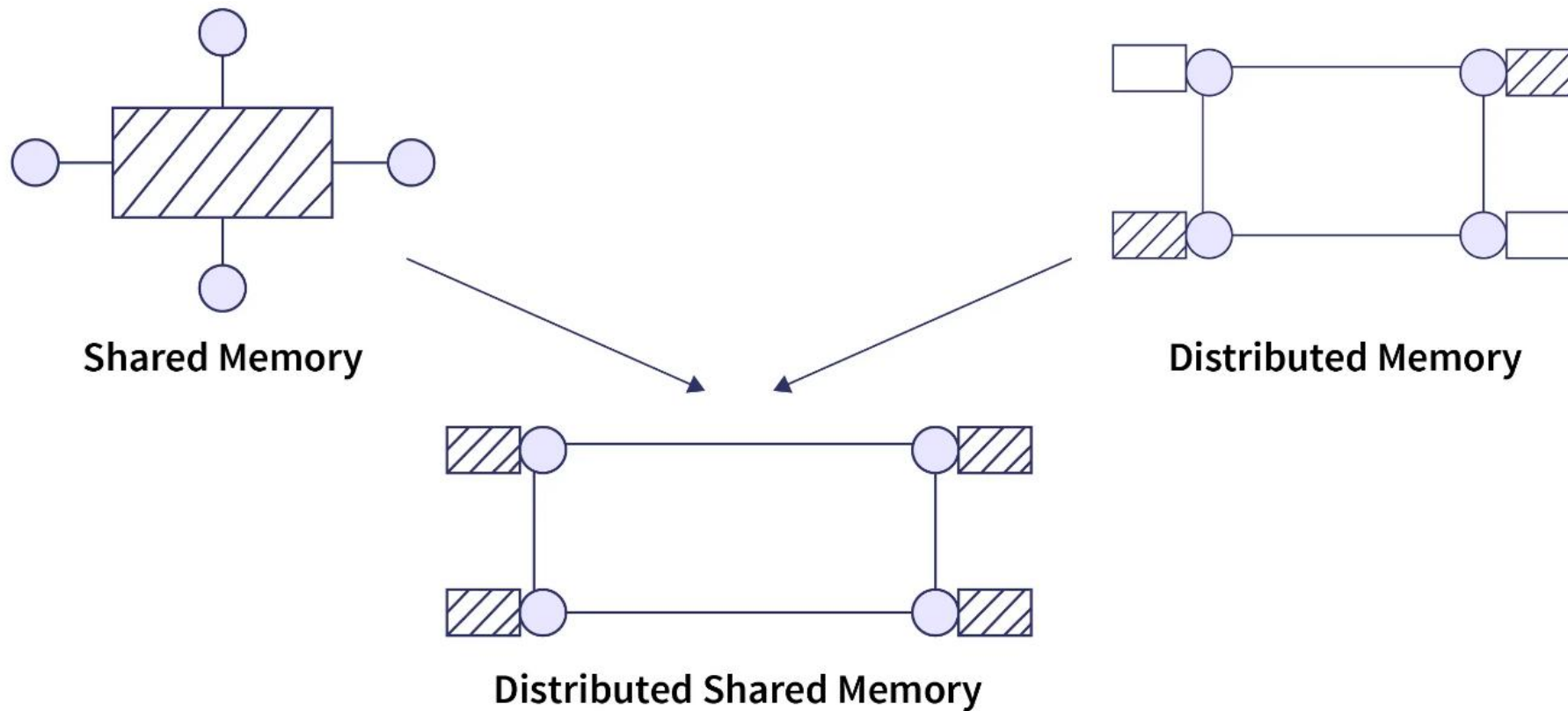
- ❑ Distributed Shared Memory
- ❑ Architecture of DSM
- ❑ Types of Distributed Shared Memory
- ❑ Methods of implementing DSM
- ❑ Software Distributed Shared Memory
- ❑ Advantages & Disadvantages of DSM
- ❑ Algorithms for DSM
- ❑ Applications of DSM



Distributed Shared Memory

- Distributed shared memory (DSM) is a form of memory architecture where *physically separated memories* can be *addressed as one logically shared address space*.
- As the word “*distributed*” is attached with the term “*shared*” so it does not mean that *there is a single centralized memory*, but that *the address space is “shared”*.
- DSM is a mechanism that manages memory across multiple nodes and makes **inter-process communications transparent** to end-users.
- Applications will operate as if they are running on a shared memory system.
- DSM is a mechanism of allowing user processes to access **shared data without using inter-process communications**.
- DSM implements the shared memory model in distributed systems, but it doesn't have physical shared memory.

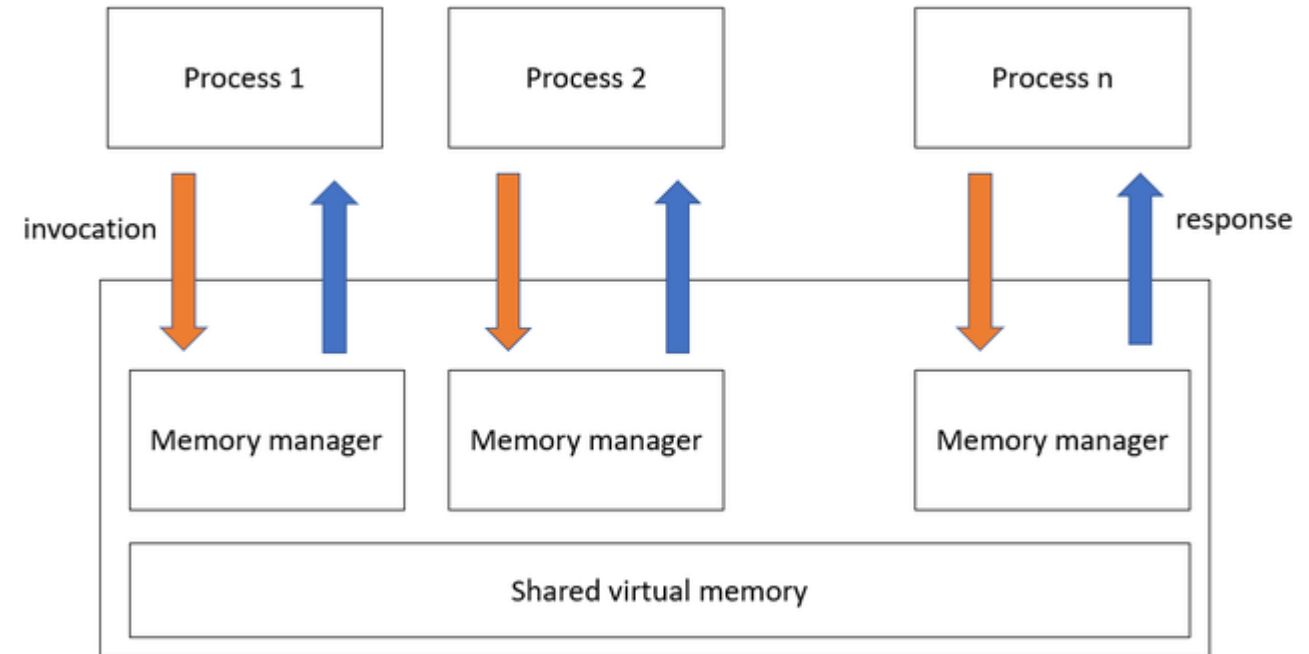
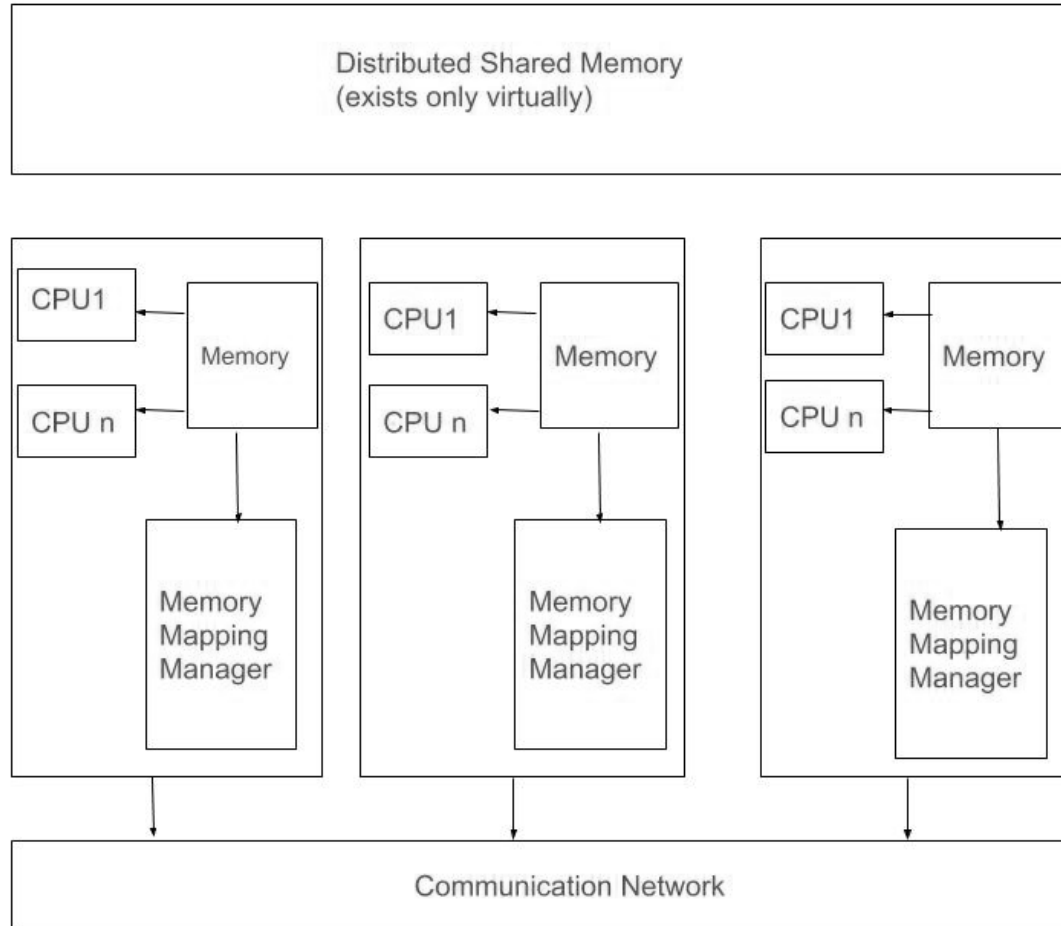
Distributed Shared Memory



Distributed Shared Memory

- Software DSM systems can be *implemented in an operating system*, or *as a programming library* and *can be thought of as extensions* of the *underlying virtual memory architecture*.
- When implemented in the *operating system*, such systems are *transparent to the developer*; whereas the *underlying distributed memory* is completely hidden from the *users*.
- In contrast, software DSM systems *implemented at the library or language level* are not transparent and developers usually *have to program them differently*.

Architecture of DSM



Types of Distributed Shared Memory

❑ On-Chip Memory

- ❑ The data resides within the CPU portion of the chip.
- ❑ Memory is directly connected to the address lines.
- ❑ This type of DSM is both expensive and complex to implement.
- ❑ Example: Modern multi-core processors (Core i5, i7, Ryzen) have Shared L3 cache

❑ Bus-Based Multiprocessors

- ❑ A set of parallel wires called a bus acts as a connection between CPU and memory.
- ❑ Accessing of same memory simultaneously by multiple CPUs is prevented by using some algorithms.
- ❑ Cache memory is used to reduce network traffic.
- ❑ Example: Traditional SMP (Symmetric Multiprocessors)

Types of Distributed Shared Memory

□ Ring-Based Multiprocessors

- ▣ There is no global centralized memory present in Ring-based DSM.
- ▣ All nodes are connected via a token passing ring.
- ▣ A single address line is divided into a shared area for distributed access.

DSM Type	How It Works	Analogy	Speed	System Size
On-Chip Memory	Shared cache on same chip	Siblings sharing the same table	Very Fast	Small (4–64 cores)
Bus-Based Multiprocessor	All CPUs share main memory via a common bus	Students sharing 1 marker	Moderate	Small to medium (2–8 CPUs)
Ring-Based Multiprocessor	CPUs connected in a ring; data travels node-to-node	Students sitting in a circle	-	Medium to large (8–32 CPUs)

Methods of implementing DSM

There are usually two methods of achieving distributed shared memory:

- **Hardware DSM**

- ▣ Such as cache coherence circuits and network interface controllers.

- **Software DSM**

- ▣ Software DSM systems implemented at the library or language level, are not transparent and developers usually have to program them differently.

Software Distributed Shared Memory

There are three ways of implementing a **software distributed shared memory**:

1. Page-based approach using the system's *virtual memory*;
2. Shared variable approach using some *routines to access shared variables*;
3. Object-based approach ideally accessing shared data through object-oriented discipline.

Page-Based Approach

- In this approach, the system's virtual memory is used to manage memory pages across multiple machines or nodes in the distributed system.
- The memory pages are shared across the system, and the DSM system handles the synchronization and coherence between them.
- **Implementation:**
 - The operating system typically handles the page-level access and synchronization.
 - When a processor accesses a page that is not locally available, a page fault occurs, and the system fetches the page from the remote memory.
- **Example:**
 - This is similar to how virtual memory works in a single machine, but applied to a distributed environment.

Shared variable approach

- This approach provides shared variables that can be accessed directly by multiple processes or threads in the distributed system.
- Instead of dealing with pages, shared variables are synchronized and managed at a higher level.
- **Implementation:**
 - ▣ The DSM system provides routines or special functions to access and update the shared variables.
 - ▣ It ensures that memory consistency is maintained by controlling how and when variables can be updated.
- **Example:**
 - ▣ Shared memory variables that can be accessed across processes in a distributed setting, often using locks or semaphores to ensure consistency.

Object-based Approach

- This approach treats shared memory as objects in an object-oriented paradigm.
- Shared data is encapsulated in objects, and the system provides methods to access and manipulate these objects in a distributed environment.
- **Implementation:**
 - The DSM system provides mechanisms to handle the distribution of objects, ensuring that objects remain consistent across multiple nodes in the system.
 - Object-oriented features like inheritance, polymorphism, and encapsulation are used to structure the shared memory.
- **Example:**
 - Distributed Object-Oriented Programming (DOOP) systems like CORBA (Common Object Request Broker Architecture) or Java RMI (Remote Method Invocation) can be used for this approach.

Comparison Methods of Software DSM

Aspect	Page-Based	Shared Variable	Object-Based
Granularity	Page-level	Variable-level	Object-level
Programming Ease	Transparent	Moderate complexity	Simplified (OO-based)
Communication Cost	High (false sharing)	Reduced	Moderate
Best for	Legacy applications	Shared-memory programming	OO-based designs

Each approach has its ideal use case, and the choice depends on application requirements and the underlying system architecture.

Advantages of Distributed Shared Memory

- **Simpler Abstraction:** Programmer need not concern about data movement, as the address space is the same it is easier to implement than RPC.
- **Easier Portability:** The access protocols used in DSM allow for a natural transition from sequential to distributed systems. DSM programs are portable as they use a common programming interface.
- **Locality of Data:** Data moved in large blocks i.e. data near to the current memory location that is being fetched, may be needed future so it will be also fetched.
- **On-Demand Data Movement:** It provided by DSM will eliminate the data exchange phase.

Advantages of Distributed Shared Memory

- ❑ **Larger Memory Space:** It provides large virtual memory space, the total memory size is the sum of the memory size of all the nodes, paging activities are reduced.
- ❑ **Better Performance:** DSM improve performance and efficiency by speeding up access to data.
- ❑ **Flexible Communication Environment:** They can join and leave DSM system without affecting the others.
- ❑ **Process Migration Simplified:** They all share the address space so one process can easily be moved to a different machine.

Disadvantages of Distributed Shared Memory

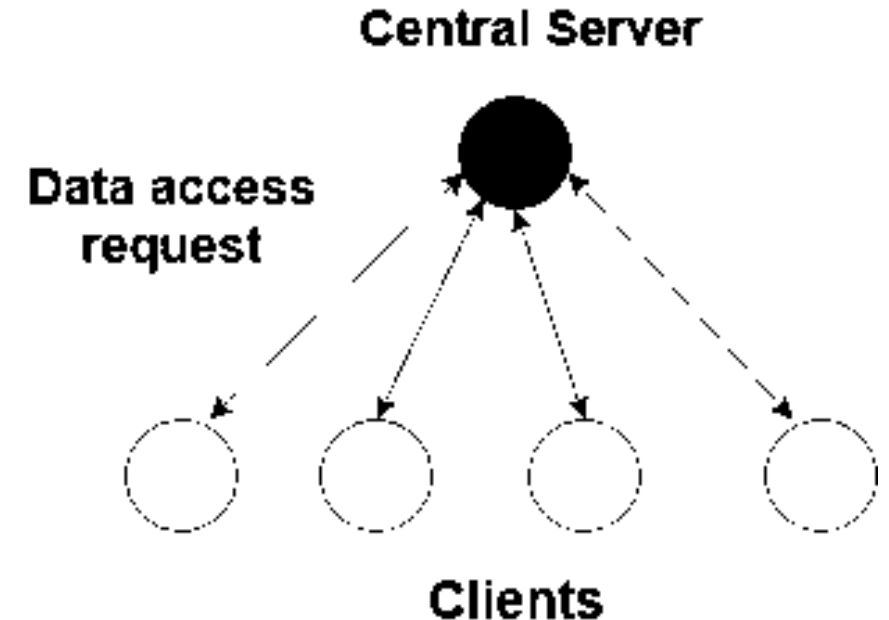
- ❑ **Accessibility:** The data access is slow in DSM as compared to non-distributed.
- ❑ **Consistency:** When programming is done in DSM systems, programmers need to maintain consistency.
- ❑ **Message Passing:** DSM use asynchronous message passing and is not efficient as per other message passing implementation.
- ❑ **Data Redundancy:** DSM allows simultaneous access to data, consistency and data redundancy is common disadvantage.
- ❑ **Lower Performance:** CPU gets slowed down, even cache memory does not aid the situation.

Algorithms for implementing DSM

- ❑ **Distributed shared memory(DSM)** system is a resource management component of distributed operating system that implements shared memory model in distributed system which have no physically shared memory.
- ❑ The shared memory model provides a virtual address space which is shared by all nodes in a distributed system.
- ❑ The central issues in implementing DSM are:
 - ▣ how to **keep track** of location of remote data.
 - ▣ how to **overcome communication overheads** and delays involved in execution of communication protocols in system for accessing remote data.
 - ▣ how to make **shared data concurrently accessible** at several nodes to improve performance.

Central Server Algorithm

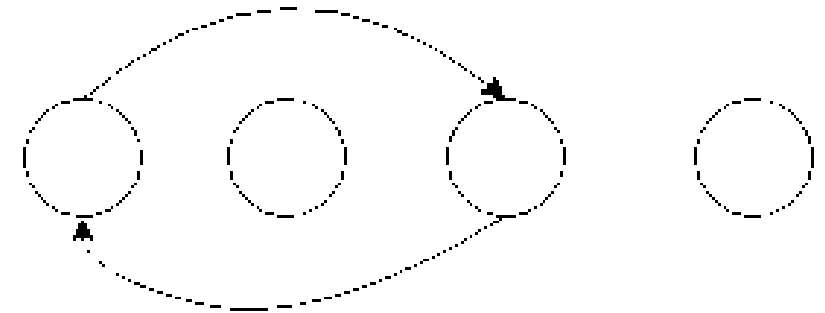
- In this, a central server maintains all shared data. It services read requests from other nodes by returning the data items to them and write requests by updating the data and returning acknowledgement messages.
- Time-out can be used in case of failed acknowledgement while sequence number can be used to avoid duplicate write requests.
- It is simpler to implement but the central server can become bottleneck and to overcome this shared data can be distributed among several servers. This distribution can be by address or by using a mapping function to locate the appropriate server.



Migration Algorithm

- In contrast to central server algo where every data access request is forwarded to location of data while in this data is shipped to location of data access request which allows subsequent access to be performed locally.
- It allows only one node to access a shared data at a time and the whole block containing data item migrates instead of individual item requested.
 - It is vulnerable to thrashing where pages frequently migrate between nodes while servicing only a few requests.
 - This algo provides an opportunity to integrate DSM with virtual memory provided by operating system at individual nodes.

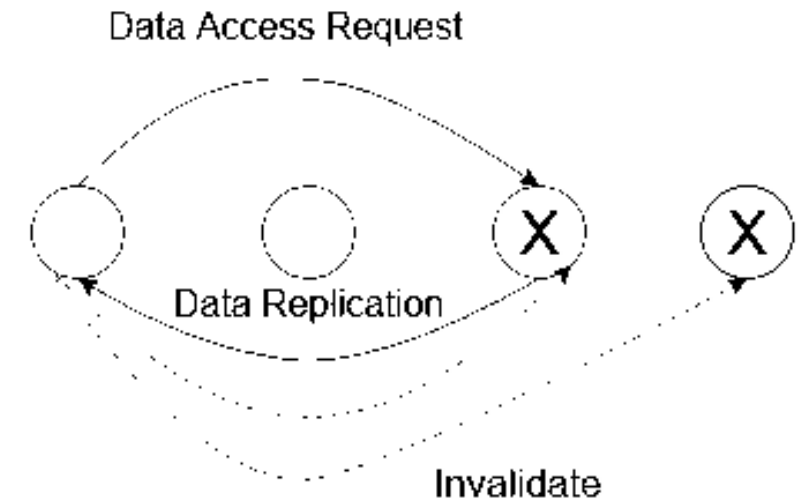
Data access request



Data block migrated

Read Replication Algorithm:

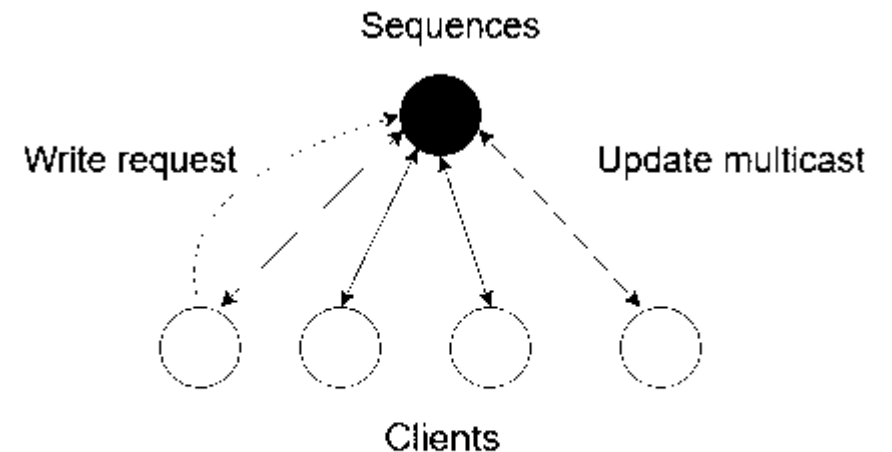
- ❑ This extends the migration algorithm by replicating data blocks and allowing multiple nodes to have read access or one node to have both read write access.
- ❑ It improves system performance by allowing multiple nodes to access data concurrently.
- ❑ The write operation in this is expensive as all copies of a shared block at various nodes will either have to be invalidated or updated with the current value to maintain consistency of shared data block.
- ❑ DSM must keep track of location of all copies of data blocks in this.



Write operation in Read Replication algorithm

Full Replication Algorithm

- In this, a central server maintains all shared data. It services read requests from other nodes by returning the data items to them and write requests by updating the data and returning acknowledgement messages.
- Time-out can be used in case of failed acknowledgement while sequence number can be used to avoid duplicate write requests.
- It is simpler to implement but the central server can become bottleneck and to overcome this shared data can be distributed among several servers. This distribution can be by address or by using a mapping function to locate the appropriate server.



Write operation in Full Replication algorithm

Applications of DSM

1. Parallel and Distributed Computing

Data-intensive applications: DSM is used in parallel applications requiring shared data, such as scientific simulations, numerical modeling, and data analytics.

Load balancing: Multiple processes can access and update shared memory, facilitating dynamic workload distribution across nodes.

Shared memory abstractions: Simplifies programming in distributed systems by providing shared memory semantics.

2. High-performance Computing (HPC)

DSM is critical in HPC environments where computation is distributed across multiple nodes but requires shared data access for efficiency.

Applications include climate modeling, fluid dynamics simulations, and protein folding analysis.

3. Database Systems

Distributed databases: DSM enables efficient data sharing and synchronization across distributed database nodes.

Caching mechanisms: Provides shared memory caching for frequently accessed data to improve performance in distributed databases.

4. Cloud Computing

Virtualization and resource sharing: DSM facilitates sharing of memory resources across virtual machines in cloud platforms.

Distributed file systems: Used for sharing metadata and enabling seamless data access across cloud storage systems.

5. Artificial Intelligence and Machine Learning

DSM is used in distributed AI and ML frameworks for sharing data models, updating weights, and coordinating tasks among worker nodes.

Thanks

25

